Artificial Intelligence Technology and Application

Machine Learning Lab Guide

Student Version

HUAWEI

Huawei Technologies CO., LTD.

Contents

1 Real-Time Recommendation Practice for Retail Products.....1
1.1 Introduction.....1
1.1.1 About This Lab.....1
1.2 Procedure.....1
1.2.1 Preparing E-commerce Platform Data.....1
1.2.2 Recommending Products Based on the Product Popularity.....6
1.2.3 Recommending Products Based on Collaborative Filtering.....10

1 Real-Time Recommendation Practice for Retail Products

1.1 Introduction

1.1.1 About This Lab

Mr. Zhao works in the AI algorithm department of e-commerce platform company A and is responsible for product recommendation for online businesses. In the modern world of the Internet and e-commerce, people are overwhelmed by data that provides useful information. However, it is impossible for users to extract the information they are interested in from the data. To help users find product information, the recommendation system can create similarities between users and products and provide suggestions for customers based on the similarities. The recommendation system is beneficial in:

- Helping users find the right products.
- Increasing user engagement. Providing recommendations. For example, Google News saw a 40% increase in hits due to recommendations.
- Helping project providers deliver projects to the right users. At Amazon, 35% of products are sold through recommendations.
- Helping personalize the recommended content. In Netflix, most rented movies are recommended ones.

1.2 Procedure

1.2.1 Preparing E-commerce Platform Data

Step 1 Import the required packages.

Functions in the NumPy library are used to perform basic operations on arrays. Pandas provides many data processing methods and time sequence operation methods.

```
# Import module packages required by the project.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2 Read data.

Step 3 Check the data overview.

View the format of the read data. You can use the **head()** function to check the first five rows of the data to get a rough understanding of the data content.

	userId	productId	Rating	timestamp
0	AKM1MP6P0OYPR	0132793040	5.0	1365811200
1	A2CX7LUOHB2NDG	0321732944	5.0	1341100800
2	A2NWSAGRHCP8N5	0439886341	1.0	1367193600
3	A2WNBOD3WNDNKT	0439886341	3.0	1374451200
4	A1GI0U4ZRJA8WN	0439886341	1.0	1334707200

Step 4 View the data size.

You can further view the data size (the number of samples and the number of features in the data) by using the **shape** function.

```
(7824482, 4)
```

Step 5 View the data type.

After learning of the data size, you still need to view the data type by using the **dtypes()** function to facilitate subsequent data calculation.

```
userId        object
productId     object
Rating       float64
timestamp      int64
dtype: object
```

According to the result, only **Rating** and **timestamp** fall into the numeric type and can be used for mathematical calculation. If **userId** and **productId** need to be used for mathematical calculation, convert the types of them. In addition, you can use the **info()** function to view the general information about the data.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7824482 entries, 0 to 7824481
Data columns (total 4 columns):
userId       object
productId    object
Rating       float64
timestamp    int64
dtypes: float64(1), int64(1), object(2)
memory usage: 238.8+ MB
```

The result contains the number of data samples, feature type, data type, and data storage size. The **info** function can display the preceding information by default, but you can set an item to **False** to hide the item. For example, you can run the following command to hide the data storage size:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7824482 entries, 0 to 7824481
Data columns (total 4 columns):
userId       object
productId    object
Rating       float64
timestamp    int64
dtypes: float64(1), int64(1), object(2)
```

Step 6 View the product ratings from users.

Product ratings are important data that can reflect users' preference. The data is critical to an efficient recommendation system. You can use the **describe** function to check the data overview of the numeric type. To view only the preliminary data analysis of **Rating**, add the corresponding column name in square brackets to the end of the command.

```
count    7.824482e+06
mean     4.012337e+00
std      1.380910e+00
min      1.000000e+00
25%      3.000000e+00
50%      5.000000e+00
75%      5.000000e+00
max      5.000000e+00
Name: Rating, dtype: float64
```

The result contains the average value, maximum value, minimum value, standard deviation, and quartile of the data, and the product rating is generally about 4. You can use the **min()** and **max()** functions to print the maximum and minimum value of the rating.

```
Minimum rating is: 1
Maximum rating is: 5
```

You can also use the **print()** function to print the result or the value of a parameter. According to the result, the highest rating is 5, indicating that users' ratings on the product are generally high.

Step 7 View the default values of the data.

The most important factors that affect data quality are default values and abnormal values. As the ratings all fall within the normal ranges, you need to use the **isnull()** function to check whether the parameter is null, and then use the **sum()** function to count the total number of non-null parameters.

```
Number of missing values across columns:
 userId        0
productId      0
Rating         0
timestamp      0
dtype: int64
```

Step 8 Check whether the users and products are unique.

A user can rate multiple products. Similarly, a product can be rated by different users. To determine the product types and the number of users, you need to check whether the users and products are unique.

```
Total data
--------------------------------------------------

Total no of ratings : 7824482
Total No of Users   : 4201696
Total No of products  : 476002
```

Step 9 Delete time information.

You can use the **drop()** function to delete the product rating time.

axis: deletes the column name part when it is set to **1**, and deletes the index number part when it is set to **0**.

inplace: indicates the operation result when it is set to **True**.

	userId	productId	Rating
0	AKM1MP6P0OYPR	0132793040	5.0
1	A2CX7LUOHB2NDG	0321732944	5.0
2	A2NWSAGRHCP8N5	0439886341	1.0
3	A2WNBOD3WNDNKT	0439886341	3.0
4	A1GI0U4ZRJA8WN	0439886341	1.0

Step 10 Analyze the rating data.

Sort the users and products by rating and view the sorting result.

groupby(): performs matching based on the specific data.

sort_values(): sorts a group of data.

ascending: ascending order.

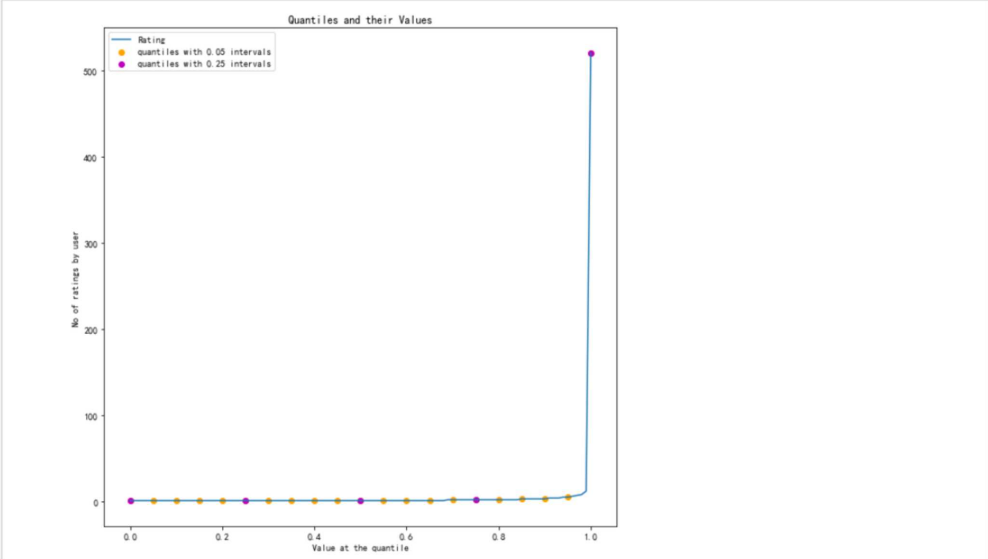
```
userId
A5JLAU2ARJ0BO      520
ADLVFFE4VBT8       501
A3OXHLG6DIBRW8     498
A6FIAB28IS79       431
A680RUE1FD08B      406
Name: Rating, dtype: int64
```

```
count    4.201696e+06
mean     1.862220e+00
std      2.885110e+00
min      1.000000e+00
25%      1.000000e+00
50%      1.000000e+00
75%      2.000000e+00
max      5.200000e+02
Name: Rating, dtype: float64
```

After obtaining the product data corresponding to the sorted user ratings, the system returns the quantiles by using the **quantile()** function, and displays the quantiles by icons.

```
# quantiles with 0.05 difference

# quantiles with 0.25 difference
```




```
No of rated product more than 50 per user : 1540
```

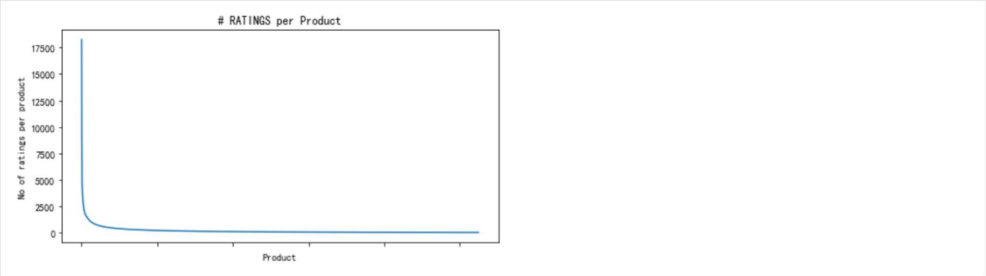
----End

1.2.2 Recommending Products Based on the Product Popularity

Sorting the products by rating to analyze the product popularity during data preparation helps implement recommendation based on the product popularity.

Step 1 Sort products.

Similar to user sorting, products can be sorted based on the rating data to obtain products that have been rated for more than 50 times.



Step 2 Sort the products by the average rating.

Calculate the average rating of each product, and then sort the products based on the average rating.

```
# Calculate the average rating of each product.
```

```
productId
0972683275    4.470980
1400501466    3.560000
1400501520    4.243902
1400501776    3.884892
1400532620    3.684211
Name: Rating, dtype: float64
```

```

```

```
productId
B002E6R7NG    4.980392
B004I763AW    4.966667
B003J9QQWU    4.964286
B0043ZLFXE    4.955556
B000TMFYBO    4.953125
Name: Rating, dtype: float64
```

```
# Obtain the rankings of the products sorted by the number of rating times.
```

```
productId
B0074BW614    18244
B00DR0PDNE    16454
B007WTAJTO    14172
B0019EHU8G    12285
B006GWO5WK    12226
Name: Rating, dtype: int64
```

```

```

productId	Rating	rating_counts
0972683275	4.470980	1051
1400501466	3.560000	250
1400501520	4.243902	82
1400501776	3.884892	139
[illegible]	[illegible]	[illegible]

The result shows that the product with the highest average rating is rated by 1051 users.

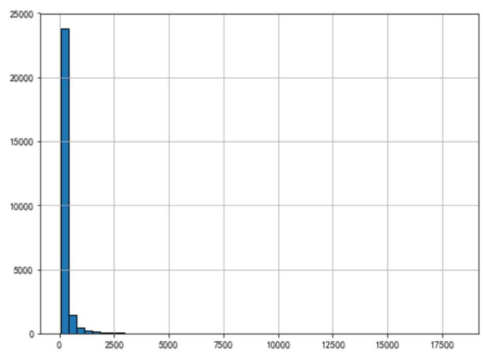
```
18244
```

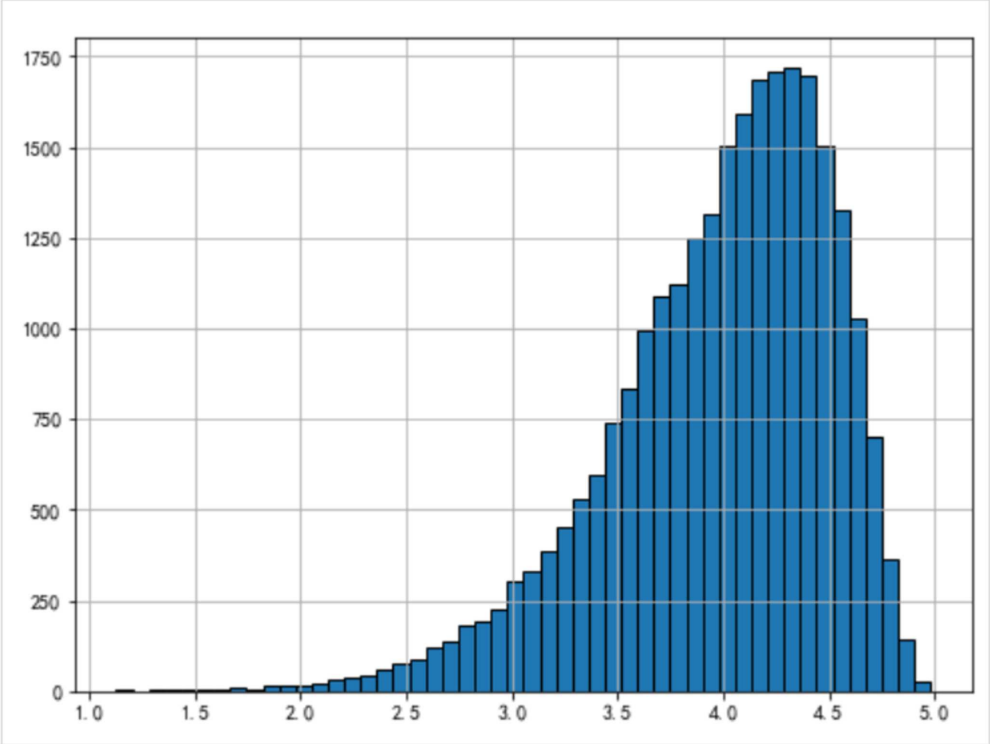
Step 3 Visualize the result.

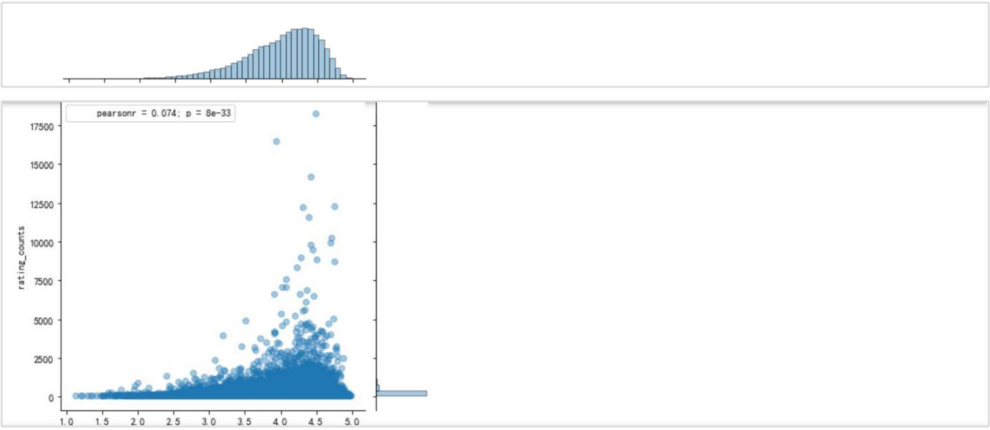
Analyze the product rankings and display the result in a chart. Specifically, use a histogram first to display the distribution of the number of users who rate each type of products.

hist(): histogram

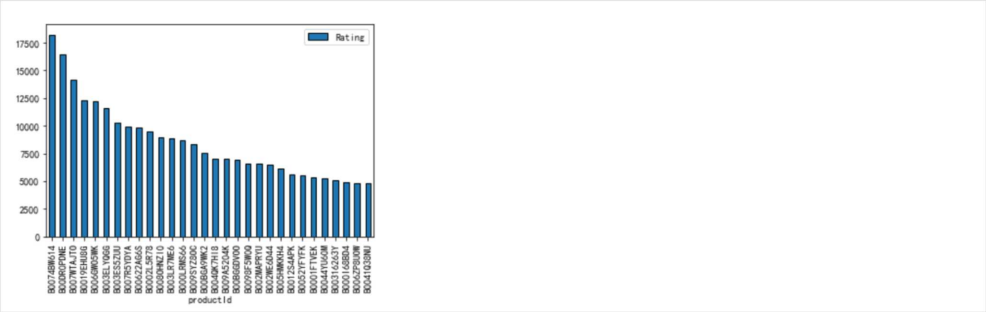
bins: number of buckets in the histogram.







Sort the products by the number of users who rate the products, to obtain the product popularity.



----End

1.2.3 Recommending Products Based on Collaborative Filtering

Recommending products based on the product popularity is not enough to meet the actual requirement. Therefore, the currently popular collaborative filtering method is used to implement recommendation.

Step 1 Create a table of relationships between products and users.

Select 10,000 samples and use **pivot_table()** to create a table of relationships between products and users.

productId userId	0972683275	1400501466	1400501520	1400501776	1400532620	1400532655	140053271X	1400532736	1400599997	1400698987	...
A01852072Z7B68UHLI5UG	0	0	0	0	0	0	0	0	0	0	...
A0266076X6KPZ6CCHGVS	0	0	0	0	0	0	0	0	0	0	...
A0293130VTX2ZXA70JQS	5	0	0	0	0	0	0	0	0	0	...
A030530627MK66BD8V4LN	4	0	0	0	0	0	0	0	0	0	...
A0571176384K8RBNKGF8O	0	0	0	0	0	0	0	0	0	0	...

5 rows × 76 columns

You can use the **shape** function to view the table size, and then transform the table. The data in the table is the product ratings from users.

```
(9832, 76)
```

userId productId	A01852072Z7B68UHLI5UG	A0266076X6KPZ6CCHGVS	A0293130VTX2ZXA70JQS	A030530627MK66BD8V4LN	A0571176384K8RBNKGF8O	A0590501PZ7I
0972683275	0	0	5	4	0	
1400501466	0	0	0	0	0	
1400501520	0	0	0	0	0	
1400501776	0	0	0	0	0	
1400532620	0	0	0	0	0	

5 rows × 9832 columns

```
# View the data size.
```

```
(76, 9832)
```

Step 2 Decompose the table.

You can use the SVD algorithm to reduce the dimensions of the table to obtain 10 important product-based features.

```
from sklearn.decomposition import TruncatedSVD # Import the SVD algorithm.
# Construct an SVD model to combine the number of features (that is, the number of columns) into 10 important combined features.
```

```
(76, 10)
```

Step 3 Build a correlation coefficient matrix.

Calculate product similarities to implement a product-based recommendation system.

corrcoef(): calculates the correlation coefficient between data.

```
(76, 76)
```

Step 4 Recommend products based on the product similarities.

Randomly select a product, select products whose coefficient of correlation with the selected one is greater than 0.65, and recommend these products to users who like the selected one.

```
# Determine whether the product is unique.
```

```
'9984984354'
```

```
# Determine whether the product is unique.
```

```
20
```

```
# View the number of similar products in the similarity table.
```

```
(76, )
```

```
#Select products whose coefficient of correlation with the 20th product is greater than 0.65.

# Delete the 20th product.

# Recommend products ranked ahead to the users who like the 20th product.
```

```
['3744295508',
 '7214047977',
 'B00000J08Q',
 'B00000J0D2',
 'B00000J0D5',
 'B00000J0D8',
 'B00000JMUG',
 'B00000JPPI']
```

As shown in the result, there are eight products whose coefficient of correlation with the 20th product (9984984354) is greater than 0.65. You can also select other products to view their similar products.

----End